Production Support Copilot

Beginner-friendly Hinglish Handbook

RAG, embeddings, vector search, LLM integration, Kafka, tools, approvals, security, evaluation and system design - explained through one runnable .NET 8 portfolio project.

LEARNING PATH	BUILD -> MEASURE -> SECURE -> EXPLAIN

Generated from the repository's canonical Markdown knowledge base.

How to use this handbook

First RAG chapters padho, then Swagger demo run karo, phir class catalog ke saath source files open karo. Interview chapter ko loud voice mein explain karo. Numbers marked as assumptions are capacity exercises, not measured production claims.

Production Support Copilot - Complete Hinglish Learning Journal

> Yeh living handbook beginner se interview-ready understanding tak le jaata hai. Code ke saath padho: har concept ke baad corresponding class open karo.

1. Hum kya bana rahe hain?

Production incident ke time engineer ko runbooks, tickets aur technical docs search karne padte hain. Normal keyword search synonyms miss kar sakta hai; plain LLM confident hallucination kar sakta hai. Hamara Copilot pehle authorized evidence retrieve karta hai, phir cited answer deta hai, current state ke liye controlled tool use karta hai, aur risky action ko human approval par rokta hai.

Example: user poochta hai, “booking status update delay kyun hai?” System tenant validate karta hai, query embedding banata hai, lexical aur vector search combine karta hai, best chunks LLM ko untrusted evidence ke roop mein deta hai, aur cited answer return karta hai.

2. Architecture ka mental model

```
flowchart LR
    U[Support Engineer] --> API[ASP.NET Core API]
    API --> AUTH[JWT + Tenant Check]
    API --> APP[Application Use Cases]
    APP --> RET[Hybrid Retriever]
    RET --> PG[(PostgreSQL + pgvector)]
    RET --> EMB[Python MiniLM Embeddings]
    APP --> LLM[LLM Provider Abstraction]
    LLM --> DEMO[Deterministic Demo]
    LLM --> OLLAMA[Optional Ollama]
    APP --> TOOLS[Controlled Tools]
    TOOLS --> APPROVAL[Human Approval]
    KAFKA[Kafka/Redpanda] --> WORKER[Idempotent Ingestion Worker]
    WORKER --> PG
```

Modular monolith kyun?

Ek deployment process hai, lekin code boundaries separate projects mein hain. Early stage mein microservices lene se network failures, distributed transactions aur deployment overhead aata. Jab independent scaling/team/security boundary ka evidence mile tab module extract karenge.

3. RAG bilkul zero se

RAG = Retrieval-Augmented Generation. LLM ki memory ko retrain nahi karte; question ke time relevant external evidence context mein attach karte hain.

Analogy: LLM ek smart student hai, RAG open-book exam hai. Student fluent ho sakta hai, par company-specific incident ka answer tab reliable hoga jab uske paas correct runbook page ho.

Pipeline:

1. **Ingestion:** document receive karo.
2. **Normalization:** line endings/format stable karo.
3. **Chunking:** long document ko meaningful pieces mein divide karo.
4. **Embedding:** har chunk ko 384 floating-point numbers mein represent karo.
5. **Indexing:** text, metadata aur vector PostgreSQL mein store karo.
6. **Retrieval:** query se lexical aur semantic candidates nikalo.

7. **Reranking/Fusion:** candidate order improve karo.
8. **Generation:** only retrieved evidence se answer banao.
9. **Citation:** claim ka source chunk return karo.
10. **Evaluation:** Recall@K, MRR, citation/abstention quality measure karo.

Embedding kya hai?

Embedding text ka numeric coordinate hai. Similar meaning wale text vectors space mein paas hote hain. `all-MiniLM-L6-v2` 384 dimensions deta hai. Vector ko map par coordinate samjho; difference sirf itna hai ki map 2D nahi, 384D hai.

Cosine similarity direction compare karti hai. Normalized vectors ke liye same direction = similar meaning. pgvector database ke andar nearest-neighbor query kar sakta hai.

Vector database kya hai?

Vector database koi magic AI database nahi. Yeh vectors store karta hai aur nearest neighbors efficiently find karta hai. Hum PostgreSQL + pgvector use karte hain because relational metadata, tenant filters, transactions aur vectors ek hi operational system mein milte hain. Early stage mein separate Pinecone/Weaviate/Qdrant ki zaroorat nahi.

Chunking kyun?

Full 50-page runbook ko prompt mein bhejna costly/noisy hai. Bahut tiny chunks context lose karte hain. Current chunker around 1200 characters with 180-character overlap use karta hai. Overlap boundary par cut hua important sentence next chunk mein preserve karta hai.

Lexical vs semantic

- Lexical: exact word match strong; IDs/error codes ke liye excellent.
- Semantic: meaning match; “falling behind” ko “consumer lag” se relate kar sakta hai.
- Hybrid: dono rankings ko Reciprocal Rank Fusion se combine karta hai.

RRF formula concept: har ranking mein position `rank` ko `1/(60+rank)` score milta hai. Dono lists mein high chunk naturally top par aata hai without incompatible raw scores normalize kiye.

Fine-tuning vs RAG

Fine-tuning model behavior/style/pattern adapt karta hai; frequently changing factual knowledge ka first solution nahi. Runbook kal change ho sakta hai—RAG index update fast hai. Fine-tuning expensive dataset, training, evaluation aur rollback discipline maangta hai. Is project mein fine-tuning intentional non-goal hai.

4. Request ka end-to-end flow

1. `POST /api/v1/auth/demo-token` JWT deta hai.
2. Caller bearer token ke saath `/copilot/query` call karta hai.
3. JWT signature, expiry, issuer, audience validate hote hain.
4. Token tenant claim request tenant se match hona chahiye.
5. Rate limiter abusive traffic control karta hai.
6. Correlation middleware trace ID set karta hai.
7. Prompt guard suspicious override phrases block karta hai.
8. Retriever only tenant-owned chunks load karta hai.

9. Query embedding + lexical scoring + vector search run hote hain.
10. RRF top passages choose karta hai.
11. Documents `<untrusted_document>` boundary mein wrap hote hain.
12. LLM/provider structured answer aur chunk IDs return karta hai.
13. Service invalid citation IDs discard karta hai.
14. Answer, citations, model, token estimates aur latency return hote hain.

5. Project-by-project responsibility

Domain

Business state aur invariants. Is project ko EF/API/HTTP ka knowledge minimum rakhna chahiye.

- `Entity`: common `Id`, created/updated timestamps.
- `Tenant`: isolation boundary; unique stable key.
- `SupportDocument`: source content, hash, classification and version.
- `DocumentChunk`: retrievable unit, offsets, token estimate, 384D vector.
- `IngestionJob`: event idempotency and lifecycle.
- `ApprovalRequest`: pending/approved/rejected/expired decision state.

Application

Use-case contracts aur orchestration.

- `DeterministicDocumentChunker`: repeatable chunks; same input gives same result.
- `IEmbeddingProvider`: model implementation ko business flow se decouple karta hai.
- `IHybridRetriever`: retrieval provider boundary.
- `GroundedCopilotQueryService`: guard -> retrieve -> generate -> citations.
- `ILlmProvider`: model/vendor abstraction.
- `ExtractiveDemoLlmProvider`: no-key safe demo; real generative model nahi.
- `ResilientLlmProvider`: primary retry, then fallback.
- `PromptInjectionGuard`: basic input pattern defense plus document trust boundary.
- Tool classes: health, lag, booking status and approval-gated replay.
- `SupportWorkflowService`: explicit state machine; model ko direct authority nahi.

Infrastructure

External technology details.

- `SupportCopilotDbContext`: EF model and PostgreSQL mapping.
- `DocumentIngestionService`: dedup, upsert, chunk, embed, persist.
- `PostgresHybridRetriever`: tenant filter, lexical/semantic ranks, RRF.
- `HttpEmbeddingProvider`: Python service HTTP adapter.
- `DeterministicEmbeddingProvider`: offline fallback, semantic ML replacement nahi.
- `OllamaLlmProvider`: local structured JSON generation.
- `KafkaIngestionPublisher`: idempotent producer configuration.

- `KafkaIngestionWorker`: manual commit and DLQ path.
- `ApprovalService`: approval persistence/decision.
- `DatabaseInitializer`: local schema + three synthetic runbooks.
- `DependencyInjection`: composition registrations.

API

Transport, authentication and middleware.

- `Program`: composition root and endpoint definitions.
- `CorrelationIdMiddleware`: request trace ID.
- `RequestTelemetryMiddleware`: bounded latency/error snapshot.

6. Data model

```
erDiagram
    TENANT ||--o{ SUPPORT_DOCUMENT : owns
    SUPPORT_DOCUMENT ||--o{ DOCUMENT_CHUNK : split_into
    TENANT ||--o{ INGESTION_JOB : processes
    TENANT ||--o{ APPROVAL_REQUEST : authorizes
    TENANT { uuid id string key string name }
    SUPPORT_DOCUMENT { uuid id uuid tenant_id string external_id string content_hash int version }
    DOCUMENT_CHUNK { uuid id uuid document_id uuid tenant_id int ordinal vector embedding }
    INGESTION_JOB { uuid id string event_id string status int attempts }
    APPROVAL_REQUEST { uuid id string action string status datetime expires }
```

Important constraints:

- `(TenantId, ExternalId)` unique: same tenant duplicate document identity avoid.
- `EventId` unique: same Kafka event process hone par duplicate chunks nahi.
- `(DocumentId, Ordinal)` unique: chunk order deterministic.
- TenantId chunk par duplicate denormalized field retrieval filter fast/obvious banata hai.

7. Kafka reliability

Producer `EnableIdempotence=true` aur `acks=all` use karta hai. Consumer auto-commit off rakhta hai: successful handling/DLQ decision ke baad offset commit hota hai. `IngestionJob.EventId` database-level duplicate defense hai. Poison item diagnostic envelope ke saath DLQ mein jaata hai.

Exactly-once marketing phrase se careful raho: end-to-end exactly-once requires coordinated transaction across broker, DB and side effects. Yahan practical **at-least-once delivery + idempotent processing** use hua hai.

8. LLM fundamentals

- **Token:** model text unit; roughly 4 English characters, language-dependent.
- **Context window:** input + retrieved context + output budget.
- **Temperature:** randomness; incident answers ke liye low value preferred.
- **Hallucination:** fluent unsupported claim.
- **Structured output:** free prose ke badle JSON contract; validation easier.
- **Streaming:** full answer wait ki jagah deltas SSE se arrive.
- **Fallback:** primary model failure par safe deterministic provider.

Default extractive provider deliberately simple hai. Optional Ollama provider JSON schema maangta hai, retries ke baad fallback hota hai, aur only supplied chunk IDs accept hote hain.

9. Agent aur workflow difference

Open-ended agent khud decide kar sakta hai next tool kya ho. Production incident remediation mein excessive agency dangerous hai. Hum explicit states use karte hain:

```
Understand -> Retrieve/DecideTool -> ExecuteTool -> Synthesize -> AwaitApproval -> Complete
```

Read-only tools directly execute hote hain. `replay_dead_letter` actual replay nahi karta; approval record create karke `AwaitApproval` par stop karta hai.

10. Security layers

- JWT signature/expiry/issuer/audience validation.
- Tenant claim vs requested tenant check.
- Retrieval query mein tenant predicate.
- Document classification filter contract.
- Suspicious prompt phrase blocking.
- Retrieved docs ko untrusted data mark karna.
- Citation allow-list: model-supplied arbitrary IDs reject.
- Consequential action requires operator role + persisted approval.
- Rate limiting and bounded tool surface.
- Synthetic data only; former employer systems/data forbidden.

Prompt injection defense ek regex se complete nahi hoti. Production mein encoding/obfuscation tests, output DLP, tool argument policy, sandboxing, least privilege, audit logs aur human review chahiye.

11. Evaluation

Golden dataset 12 categories cover karta hai: answerable, unanswerable, injection, authorization, staleness, tool and approval.

- Recall@1: expected doc first result hai?
- Recall@5: expected doc top five mein hai?
- MRR: correct result jitna upar, score utna high.
- Grounded citation rate: answerable query cited evidence ke saath grounded thi?
- Safe abstention rate: unanswerable/unsafe query par system ne guess avoid kiya?
- p50/p95 latency: typical and tail response time.

Tiny seed corpus ka perfect score production proof nahi. Dataset size, environment, commit aur date always report karo.

12. Observability

Correlation ID request logs join karta hai. Response model latency/token estimates expose karta hai. In-memory dashboard last 500 requests se count, errors, p50/p95 dikhata hai. Production path: OpenTelemetry traces ->

collector -> Grafana/Tempo/Jaeger; metrics -> Prometheus; logs -> centralized store.

13. Docker topology

- `postgres`: relational data + pgvector.
- `redpanda`: Kafka-compatible local broker.
- `embeddings`: Python FastAPI + MiniLM.
- `api`: .NET orchestration.

Compose enables real embeddings and Kafka. Direct `dotnet run` defaults are lightweight: deterministic embedding + direct ingestion.

14. Run and demo

```
docker compose up --build
```

Open `http://localhost:5051/swagger`. First demo token lo. Swagger `Authorize` mein `Bearer <token>` enter karo. Then:

1. Hybrid search: `Why are booking status updates delayed?`
2. Copilot query: citations verify karo.
3. Workflow: `What is current health of booking API?` read-only tool trace dekho.
4. Workflow: `Replay the DLQ` approval ID dekho.
5. Approval decision endpoint operator token se call karo.
6. Injection query try karo; block expected.
7. Another tenant request try karo; 403 expected.

15. Interview explanation - 90 seconds

“I built a multi-tenant production support copilot as a .NET 8 modular monolith. Documents enter through a Kafka-compatible ingestion path; an event-id unique constraint makes consumers idempotent and failures go to a DLQ. Content is deterministically chunked and embedded by a small Python MiniLM service, then stored with metadata in PostgreSQL/pgvector. Serving uses tenant-filtered lexical and vector retrieval fused with RRF. The LLM is behind an abstraction, returns structured cited output, and falls back safely. Current state is fetched through allow-listed read-only tools, while consequential actions pause at a persisted human approval. JWT tenant checks, prompt-injection boundaries, rate limits, evaluation data, traces/latency and Docker/CI assets make reliability measurable.”

16. What to learn next

1. PostgreSQL full-text `tsvector` + GIN and HNSW indexing under larger corpus.
2. Real OpenTelemetry exporter/backend.
3. Durable workflow/outbox for approval completion.
4. Object storage and PDF/HTML parsers.
5. Reranker cross-encoder and retrieval experiments.
6. Production JWT identity provider and document ACL tables.
7. Load testing, SLOs and capacity measurements.
8. Cloud deployment with managed Postgres/Kafka/secrets.

17. Honest completion checklist

- Core architecture and all requested concern paths are represented in runnable code.
- Paid cloud LLM is not required; Ollama is optional.
- Real employer integrations are deliberately absent.
- A local demo still needs Docker images/model download on first run.
- “Production-minded portfolio” does not mean “audited production system.”

Class and Service Catalog

Related: 01 Architecture Map, 02 RAG From Zero, 05 Runbook and Demo.

Domain classes

Class	Kya rakhta hai	Important behavior/invariant
`Entity`	ID and timestamps	`MarkUpdated` state change time refresh karta hai
`Tenant`	stable key and display name	empty values reject; documents ownership root
`SupportDocument`	source text, hash, version, classification	replacement hash/version update; tenant+external ID unique
`DocumentChunk`	ordinal, offsets, text, token estimate, vector	exactly 384 dimensions enforce
`IngestionJob`	event ID, attempts, status, error	duplicate event tracked; state transitions explicit
`ApprovalRequest`	action, arguments, requester, expiry/decision	one decision only; expiry check

Application contracts and services

Class/interface	Role	Why separate?
`ChunkDraft`	chunker output value	EF entity create hone se pehle pure data
`IngestDocumentCommand`	ingestion message contract	HTTP/Kafka same contract use karte
`IDocumentChunker`	chunk algorithm boundary	alternative token-aware chunker testable
`DeterministicDocumentChunker`	paragraph/sentence boundary chunks	repeatable indexing/evaluation
`IDocumentIngestionService`	end-to-end indexing use case	API/consumer implementation details se free
`IIngestionEventPublisher`	async boundary	direct learning mode vs Kafka mode
`RetrievalQuery`	tenant/query/topK/filter	authorization-aware search input
`RetrievedPassage`	ranked evidence	raw component + combined scores observable
`IEmbeddingProvider`	text -> float vector	model/service swappable
`IHybridRetriever`	query -> passages	Postgres/search implementation swappable
`IPromptInjectionGuard`	inspect and mark trust	safety policy centralized
`PromptInjectionGuard`	basic phrase checks + boundary tags	defense-in-depth first layer
`LlmRequest`	question + evidence + system policy	clear model contract
`LlmAnswer`	answer + cited IDs + tokens	structured validation and metrics
`ILlmProvider`	model abstraction	vendor/local fallback
`ExtractiveDemoLlmProvider`	deterministic evidence summary	free, stable tests/demo
`ResilientLlmProvider`	3 retries then fallback	graceful degradation
`ICopilotQueryService`	public answer use case	endpoint thin rakhta hai
`GroundedCopilotQueryService`	guard/retrieve/generate/validate citations	main RAG orchestration
`ISupportTool`	named controlled capability	allow-list and policy metadata

Class/interface	Role	Why separate?
`ToolRegistry`	tool lookup	model/user arbitrary code execute nahi kar sakta
`ServiceHealthTool`	mock live health	docs vs live-state distinction
`ConsumerLagTool`	mock Kafka lag	read-only operational evidence
`BookingStatusTool`	synthetic booking lookup	safe domain-relevant demo
`ReplayDeadLetterTool`	approval request only	consequential action boundary
`IApprovalService`	request/decision persistence contract	workflow database decoupling
`SupportWorkflowService`	explicit routing/state trace	agent decisions auditable/deterministic

Infrastructure classes

Class	External responsibility	Failure behavior
`SupportCopilotDbContext`	EF tables, constraints, vector extension	DB exceptions surface to API/logs
`DatabaseInitializer`	local create + seed runbooks	startup fails fast if configured DB unavailable
`DocumentIngestionService`	idempotency, upsert, chunk, embed, persist	job error captured; DLQ path can publish
`DeterministicEmbeddingProvider`	offline hashed vectors	always local; not true semantic model
`HttpEmbeddingProvider`	call Python `/embed`	timeout/status/dimension validation
`PostgresHybridRetriever`	tenant-safe lexical+semantic+RRF	unknown tenant returns empty evidence
`OllamaLlmProvider`	local model structured JSON	invalid HTTP/JSON triggers resilient fallback
`KafkaOptions`	broker/topic/group config	environment override friendly
`KafkaIngestionPublisher`	keyed idempotent publication	acks-all/retries; caller sees delivery error
`DirectIngestionPublisher`	no-broker learning path	executes ingestion in request path
`KafkaIngestionWorker`	consume, process, DLQ, manual commit	transient loop retry; poison diagnostic envelope
`ApprovalService`	approval DB operations	missing/expired/already decided rejected
`DependencyInjection`	Infrastructure registrations	one composition method avoids scattered wiring

API classes

Class	Role	Detail
`Program`	composition root	Swagger, JWT, rate limits, endpoints, startup seed
`CorrelationIdMiddleware`	trace header/scope	accepts supplied ID or creates one
`RequestTelemetry`	bounded recent metrics	500 item queue, p50/p95 snapshot
`RequestTelemetryMiddleware`	measures every request	finally block records failures too
`DemoTokenRequest`	local auth request	default support/operator roles
`ToolExecutionRequest`	JSON tool arguments	tool validates expected properties
`ApprovalDecisionRequest`	approve/reject input	operator role enforced endpoint-side

Python classes/functions

Symbol	Role
`EmbedRequest`	1-64 input texts validation
`EmbedResponse`	vectors, model and dimensions
`model()`	cached SentenceTransformer load
`embed()`	normalized batch embedding endpoint
`evaluate.py post()`	dependency-free HTTP helper
`evaluate.py main()`	golden dataset metrics/report

Security Reliability and Evaluation

Related: 02 RAG From Zero, 06 System Design Interview, 07 Glossary.

Threat path

```
flowchart LR
    Input --> JWT --> TenantCheck --> PromptGuard --> RetrievalACL --> UntrustedBoundary --> CitationAllowList --> Output
    ToolCall --> RoleCheck --> Approval --> Audit
```

Tenant isolation must happen before LLM context. Output filtering after leakage is too late. Prompt injection is untrusted content attempting to become instruction; system treats retrieved text as data and never gives it policy authority.

Reliability path: correlation ID, bounded retries, timeouts, deterministic fallback, Kafka manual commit, idempotency unique constraint, DLQ, safe abstention and evaluation regression.

Quality path: golden examples -> retrieval ranks -> grounded citations -> safe abstention -> latency/cost -> release threshold. Human review remains necessary for incident advice.

Runbook and Demo

Related: 03 Class and Service Catalog, 06 System Design Interview.

Quick run

```
docker compose up --build
```

Swagger: `http://localhost:5051/swagger`

Token -> Authorize -> hybrid query -> grounded query -> current-health workflow -> DLQ replay workflow -> approval -> injection/tenant negative tests.

Database inspection

pgAdmin connection: host `localhost`, port `5432`, database `support_copilot`, user `support`, password `support_dev_password`. Tables live under Schemas -> public -> Tables. View `tenants`, `support_documents`, `document_chunks`, `ingestion_jobs`, `approval_requests`.

Evaluation

```
python evaluation/evaluate.py --base-url http://localhost:5051
```

Common failures

- Port 5432 in use: stop old Postgres container or change compose host port.
- First embedding build slow: PyTorch/model download is large; model cache volume makes later starts faster.
- Swagger 401: obtain token and click Authorize.
- 403: token tenant and request tenant differ, or operator role missing.
- Empty retrieval: seed did not complete; inspect API logs and database.
- Locked DLL on Windows: stop running API/testhost, then build again.
- NuGet NU1900 offline warning: vulnerability feed unreachable; build may still compile from cache, but CI should perform online audit.

System Design Interview

Related: 01 Architecture Map, 04 Security Reliability and Evaluation, 05 Runbook and Demo.

Capacity example (assumptions, not measured production facts)

Assume 100 tenants, 10,000 documents each, average 20 chunks = 20 million chunks. A 384D float32 vector is about 1.5 KB raw, roughly 30 GB raw vectors before table/index overhead. If 10 QPS and top-50 candidates, exact scan is no longer acceptable; partition/tenant filters plus HNSW and connection pooling become relevant. Embedding ingestion needs backpressure and autoscaled consumers. LLM concurrency is bounded by provider rate limits and cost budgets.

Interview tradeoffs

- Consistency: document transaction completes before job complete; Kafka gives at-least-once.
- Availability: LLM failure falls back; DB failure prevents grounded answer rather than hallucinating.
- Latency: parallel lexical/vector candidates can reduce time; reranker adds quality and latency.
- Cost: small embedding model, Postgres consolidation, prompt context cap, cached/deduplicated ingestion.
- Security: tenant filter before context, role-gated tools, approval, secrets outside source.
- Scale: start modular monolith; extract ingestion/model gateway on evidence.

Failure questions

If embedding service down: ingestion fails/retries/DLQ; existing indexed docs remain searchable. If LLM down: deterministic cited fallback. If duplicate Kafka message: unique EventId returns duplicate result. If malicious document: trust boundary, no tool authority, citation allow-list, tests. If approval service restarts: persisted request survives; current implementation decision endpoint completes simulated action only, while production needs durable continuation/outbox.

Glossary

Related: 02 RAG From Zero, 04 Security Reliability and Evaluation.

- **LLM:** next-token model capable of language tasks.
- **Embedding:** fixed-size numeric representation of meaning.
- **Vector:** ordered list of numbers.
- **Cosine distance:** angular difference between vectors.
- **RAG:** retrieve external evidence before generation.
- **Chunk:** independently indexed document segment.
- **Hybrid retrieval:** lexical plus semantic search.
- **Reranker:** second-stage candidate ordering model/algorithm.
- **Grounding:** tying claims to provided evidence.
- **Hallucination:** unsupported generated claim.
- **Citation:** source identifier attached to answer claim/evidence.
- **Prompt injection:** untrusted input attempting to override instructions.
- **Tool calling:** model/workflow selecting a controlled application function.
- **Agent:** iterative model-directed workflow; use bounded authority.
- **State machine:** explicit allowed stages/transitions.
- **Idempotency:** retry produces no duplicate effect.
- **DLQ:** dead-letter queue for poison messages.
- **RRF:** reciprocal rank fusion of ranked lists.
- **Recall@K:** fraction where expected result appears in top K.
- **MRR:** mean reciprocal rank; rewards earlier correct result.
- **p95 latency:** 95% requests at or below this duration.
- **Fine-tuning:** changing model weights with task examples.
- **Structured output:** response constrained to schema.
- **SSE:** server-sent events for one-way streaming responses.